Module 4 Tangent:

Functional Programming

Not to be confused with Dysfunctional Programming (OOP).

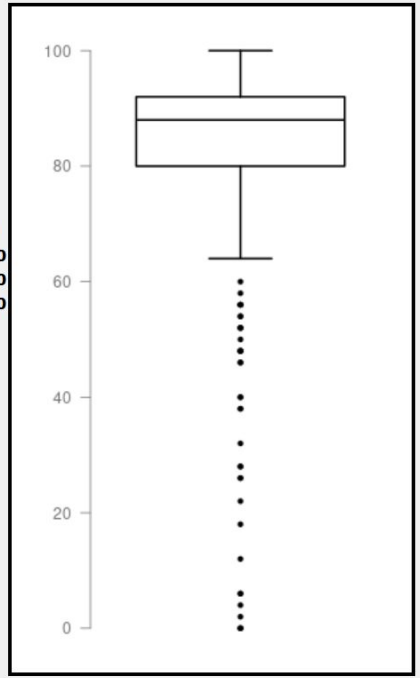
Evaluator Test 1

Average 80.64%, Median 88%

- Overall good, many compilation issues though.

Population size: 276
Median: 88
Minimum: 0
Maximum: 100
First quartile: 80
Third quartile: 92
Interquartile Range: 12
Outliers: 0.00 0.00 0.00 2.00 4.00 6.00 6.00 12.00 18.00 22.00 26.00 26.00 28.00 28.00 32.00 38.00 38.00 40.00 40.00 46.00 46.00 48.00 48.00 48.00 48.00 50.00 52.00 52.00 52.00 54.00 54.00 56.00 56.00 56.00 56.00 58.00 60.00

Mean
Geometric mean
Harmonic mean
Median
Mode
Standard Deviation
Variance
Mean absolute deviation
Range
Interquartile range
Quartiles
All dispersion data



0. Motivation

Only C++ or nullptr!

Why learn other paradigms?

1. Knowing multiple approaches helps understanding.

2. Languages adopt ideas from other paradigms (FP)

C ‘72

C++ ‘85

C# ‘00

Python ‘90

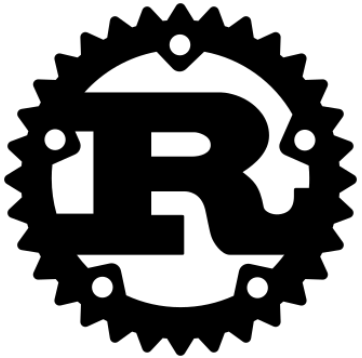
Ruby ‘95

Java ‘95

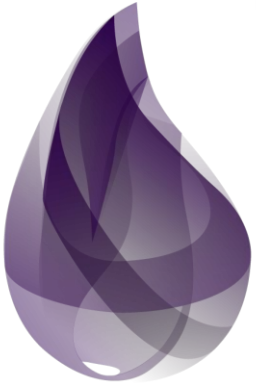
JavaScript ‘95

MatLab ‘84

Swift ‘14



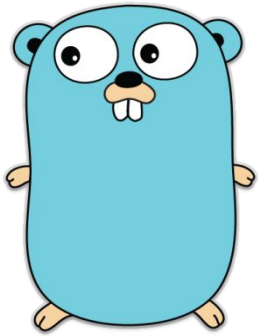
Rust ‘10



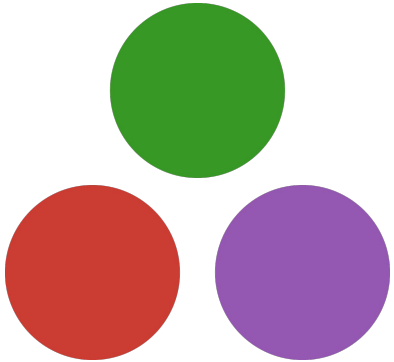
Elixir ‘12

Elm ‘12

Kotlin ‘11



Go ‘09



Julia ‘12

Zig ‘16

1.
Definitions

“What are we talking about again?”

Definition

It's About Functions!

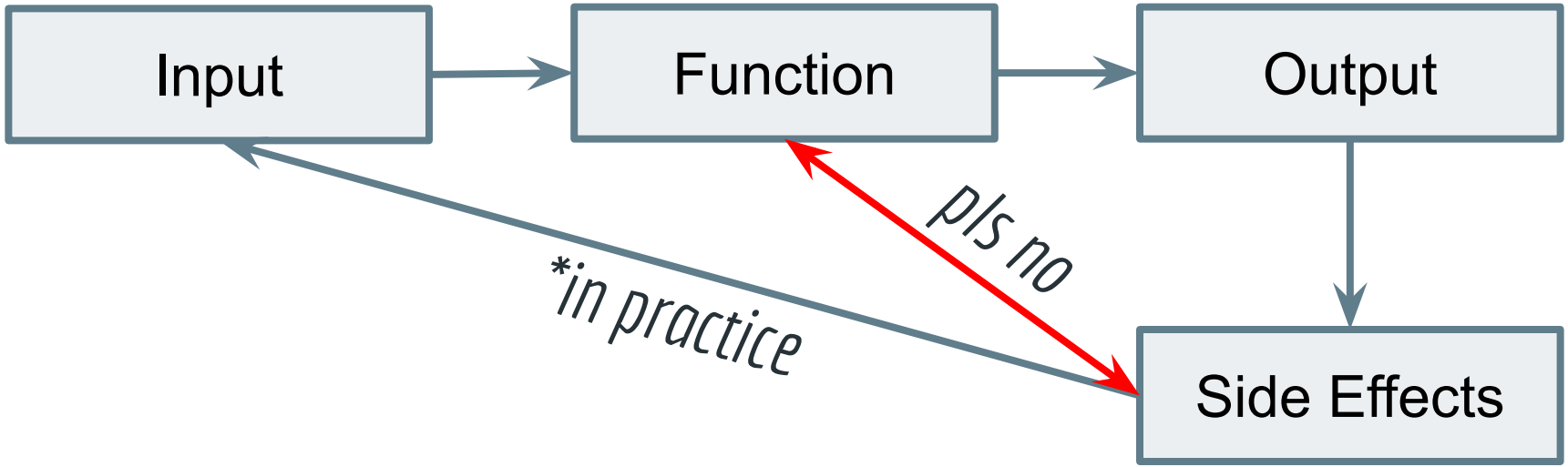
FP is a paradigm based around the composition of mathematical functions.

- Mathematical functions are pure:
  - Same input, same output.
  - No mutation or side effects.
- Pure functions are easy to *compose*.
  - No magic hidden/mutable state!
  - Large problems can be broken down into many small functions.

Theory* of FP

Observation: Computation is heavily based around transformations of data.

- Functions that transform data can be *composed* into larger transformations.
  - `a → b + b → c = a → c`
  - Clear data flow from start → end.
- Observation: Mutability hurts composition!
  - Mutations & side effects require manually tracking underlying state.



The “Ideal” Functional Program

Functional Code

“Markers” of FP

- First-class & higher-order functions:
  - Functions can be stored and passed around as values (like lambdas)
- Immutability & purity:
  - Data is immutable, functions should be “pure” and only transform data.
  - Hence: recursion, not iteration.
- Type systems & type inference:
  - Very powerful type systems
  - Types exist, even if not explicit!

```
func map([], _): = [];
func map([head, tail*], f) =
  [f(head)] + map(tail, f);

map([1, 2, 3], square)
  == [1, 4, 9]
```

1.

Immutability & Purity

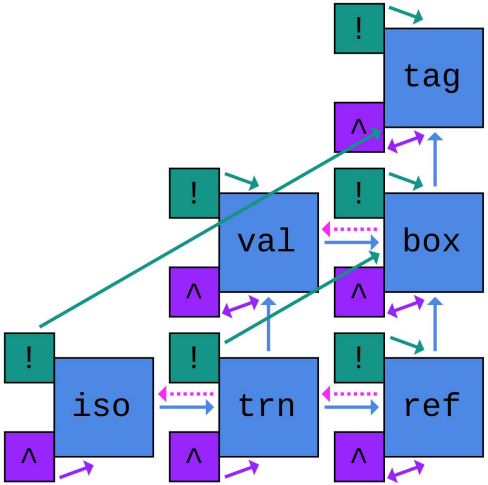
“What are we talking about again?”

Immutability

All (Most) Data is Immutable!

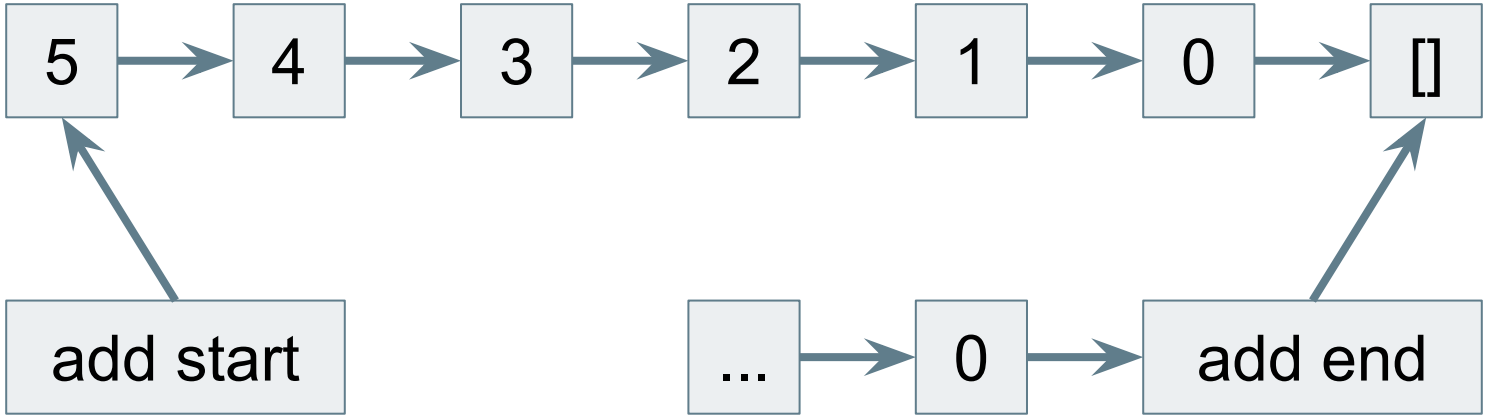
Functional languages emphasize immutable data, limiting the risk of unexpected mutation.

- Reduces the complexity of mutable state.
- Data can be shared and reused freely.
  - Great for appending/transforming data.
  - Performance hit when “mutating” (aka rebuilding) entire data structures.
- Most languages provide higher-level constructs for managing mutable state.



Immutability > This

type List<T> = [] or (T, List<T>)



Cons List

Purity

Definition

Purity is a measure of how isolated a function is from external state.

- **Pure** functions depend only on input and have no side effects.
  - Easy to think about and compose.
  - A related term is **referentially transparent**, which is… messy.
- Might be better to think about specific state/effects (DB, Files, etc.)

Purity + Mutability?

Mutability isn't *incompatible* with purity, but does add additional nuance.

- "Internal" mutations are only part of the impl. and can't be observed externally.
- "Local" mutations may modify local state, e.g. the method receiver or arguments.
  - Reads (e.g. getters) are more "pure".
- "Global" reads/writes are more complex.
  - Reading global state (Environment?)
  - Writing to the file system (DB? Logging?)

Is It Pure?

The hot new show that no one watches!

```java
int function(int a, int b) {
  return a + b;
}
//yes, definitely pure!

void function() {
  System.out.println("Hello!");
}
//no, definitely not pure!

void function(List<Integer> list) {
  list.add(0);
}
//kinda, pure considering list state
```

```java
record Data(int a) {
  int method(int b) { return a + b; }
}
//yes, pure for same instance

class Data {
  int a;
  int method(int b) { return a + b; }
}
//yes, pure for same input state

class Data {
  int a;
  void method(int b) { a = b; }
}
//kinda, pure considering obj state
```

Is It Pure?

Module 4 Tangent:

Functional Programming

AKA Evaluator Part II: Electric Boogaloo

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals